

Recurrent Neural Network (RNN)

Recurrent Neural Network (RNN) is a type of Deep Learning neural network specially designed for **Sequential Data**, where the order of information matters.

Unlike ANN, RNN remembers previous information while processing the current input, making it suitable for text, speech, and time-series data.

What is Sequential Data?

Sequential Data is data in which the order of elements is important.

Example:

Age	Location	Salary
24	-	1.5 Lakhs
27	-	6.5 Lakhs

In Neural Networks (ANN), we can swap the inputs without changing the learning process. This is called **Non-Sequential Data**.

However, in text:

"I am happy"

and

"Happy am I"

contain the same words but different meanings because order matters.

Therefore, text is **Sequential Data**, and we use **RNN** to process it.

Why Use RNN Instead of ANN?

Although ANN can process text, it has several limitations.

1. Variable Input Length

Text sentences have different numbers of words.

ANN expects fixed-size inputs.

2. High Computational Power

Each word creates additional weights.

Example:

21 Inputs

3 Perceptrons

$21 \times 3 = 63$ weights

3. Out Of Vocabulary (OOV)

ANN struggles when completely new words appear.

4. Semantic Meaning is Lost

The biggest problem is that ANN cannot preserve the sequence of words.

Example:

"I am not happy"

"I am happy"

ANN cannot properly understand the difference because sequence is not retained.

Applications

- Sentiment Analysis
- Text Classification
- Language Modeling
- Speech Recognition
- Machine Translation

(Today, simple RNNs are used much less because LSTM and GRU perform better.)

Data Feeding in RNN

Before feeding text into RNN, words must be converted into numerical vectors.

Example:

You are good → 1

You are bad → 0

You are not good → 0

Vocabulary:

You

Are

Good

Bad

Not

Each word is converted into vectors and then fed sequentially into the RNN.

Keras Input Parameters

- Batch Size = 3
- Time Steps = 4 (maximum words in a sentence)
- Input Features = 5 (Vocabulary Size)

RNN Architecture

RNN is called a **Feedback Neural Network** because information from previous hidden states is passed back into the network.

ANN Architecture

Input



Hidden Layer



Output

This is called a **Feed Forward Network**.

Information only moves forward.

RNN Architecture

Input



Hidden Layer

↺ (Feedback Loop)



Output

The hidden layer sends information back to itself.

This memory helps RNN understand sequences.

Example

5 Inputs



3 Hidden Neurons



1 Output

ANN

$5 \times 3 = 15$ Weights

+3 Output Weights

RNN

Hidden neurons communicate with each other repeatedly.

$3 \times 3 = 9$ Recurrent Weights

Total = 27 Weights

Example:

You



Are



Good

Each word updates the hidden state.

Hidden State

The **Hidden State** is the memory of the RNN that stores information learned from previous words.

It allows the network to remember previous context while processing the next word.

Activation Function

The default activation function inside the hidden layer of RNN is:

tanh

It keeps values between:

-1 and +1

Mathematical Flow

(Input $\times$ Weight)



Hidden State



Output

First Hidden State

$(\text{You} \times W_i)$

↓

O_1

Second Hidden State

$f(\text{Input} \times W_i)$

+

$(\text{Hidden Weight} \times \text{Previous Output})$

↓

O_2

This loop continues for:

O_1

↓

O_2

↓

O_3



...

until the complete sequence is processed.

Simple Definition of RNN

A Recurrent Neural Network is a neural network where hidden neurons repeatedly interact with previous hidden states, allowing the model to remember earlier information while processing sequential data.

Problems with RNN

Although RNN introduced memory, it still has several limitations.

1. Weak Long-Term Memory

Cannot remember information for long sequences.

2. Vanishing Gradient Problem

Gradients become extremely small during backpropagation.

Learning almost stops.

3. Exploding Gradient Problem

Gradients become extremely large.

Training becomes unstable.

4. Long Dependencies are Lost

Important words appearing much earlier are forgotten.

5. Sequential Processing

Words are processed one by one.

Training becomes slow.

Solution

Because of these problems,

LSTM and GRU were developed.

They use special gates to decide:

- What to remember
- What to forget
- What to update

This allows them to capture long-range dependencies much better.

LSTM (Long Short-Term Memory)

LSTM is an improved version of RNN designed to solve the long-term memory problem using a memory cell and three special gates.

Main Components

C_{t-1}



Previous Cell State (Long-Term Memory)

H_{t-1}



Previous Hidden State (Short-Term Memory)

X_t



Current Input

C_t



Current Cell State

H_t



Current Hidden State

Cell State and Hidden State always have the same size because element-wise operations are performed on them.

Forget Gate

The **Forget Gate** decides which information from the previous cell state should be kept or removed.

Uses:

Sigmoid Activation Function

Output:

0 → Forget

1 → Keep

Formula:

$$F_t = \sigma(W_f[H_{t-1}, X_t] + B_f)$$

Example

Riya is a Doctor.

She lives in Delhi.

She works at a Hospital.

Last year she moved to London.

Now she works at a Research Lab.

Forget Gate assigns importance:

Gender → 0.99

Profession → 0.95

Delhi → 0.10

Hospital → 0.85

London → 0.90

Old information gradually fades instead of disappearing instantly.

Input Gate

The **Input Gate** decides what new information should be added to the Cell State.

Uses:

- . Sigmoid
- . tanh

Main Components

C'_t

Candidate Cell State

I_t

Input Gate

Cell State Update

$$C_t = F_t \times C_{t-1}$$

+

$$I_t \times C'_t$$

Output Gate

The **Output Gate** decides what information should be sent to the next hidden state.

Uses:

- Sigmoid
- tanh

Main Equation

H_t

=

$$O_t \times \tanh(C_t)$$

LSTM Architecture

Every timestamp contains:

- Forget Gate
- Input Gate
- Output Gate

Along with:

- Cell State (Long-Term Memory)
- Hidden State (Short-Term Memory)

Workflow

Forget Gate



Removes unnecessary information



Input Gate



Adds useful information



Output Gate



Generates new Hidden State



Passes information to next timestamp

GRU (Gated Recurrent Unit)

GRU is a simplified version of LSTM that solves long-term dependency problems using only **two gates**, making it faster and computationally efficient.

Why GRU?

Problems with LSTM:

- Complex Architecture
- More Parameters
- Slower Training

GRU simplifies all of these.

Main Components

H_{t-1}

Previous Hidden State

X_t

Current Input

H_t

Current Hidden State

R_t

Reset Gate

Z_t

Update Gate

H'_t

Candidate Hidden State

Uses:

- Sigmoid Activation
- tanh Activation

Update Gate

The **Update Gate** decides how much previous memory should be retained.

Example

High Value

$Z_t = 0.9$

Keep old information.

Low Value

$$Z_t = 0.1$$

Replace old information.

Example:

Old $\rightarrow$ Delhi



New $\rightarrow$ London

Reset Gate

The **Reset Gate** decides how much past information should influence the new information.

After calculating:

Reset Gate

.

Update Gate



Candidate Hidden State



Current Hidden State

Example

Riya lives in Delhi.



Riya moves to London.

GRU:

Creates new information



Mixes with previous memory



Update Gate decides the mixing ratio



Hidden State is continuously updated.

Complete Connection

Text Data



Order matters



Need Sequential Model



RNN remembers previous words



RNN suffers from Vanishing Gradient & Weak Memory



LSTM introduces:

- Forget Gate
- Input Gate

- Output Gate



Better Long-Term Memory



GRU simplifies LSTM



Only Reset Gate + Update Gate



Faster Training with similar performance.